

CUCEI

Reporte de actividad:

ALU

Arquitectura de computadoras

sección D12

Jorge Ernesto Lopez Arce Delgado

María del Sagrario Lomelí Mejía

Ingeniería informática (INFO) 224778592

16 de febrero de 2026.

Introducción e historia de la ALU

El desarrollo de la Unidad Aritmético-Lógica (ALU) se remonta a las ideas de Charles Babbage, quien conceptualizó las primeras máquinas con programa almacenado. En su propuesta ya se distinguían componentes fundamentales como la unidad de control, la memoria y una unidad aritmética, bases que posteriormente se utilizarían en las primeras computadoras.

Durante la década de 1950 surgieron máquinas como la computadora Mosaic (1953), que utilizaba más de 6480 válvulas electrónicas y ocupaba grandes espacios físicos. En este contexto, el "Arithmetic Rack" representó una de las primeras implementaciones de una unidad aritmética. Más adelante, la EDSAC 2 (1958) introdujo el uso de unidades microprogramadas, marcando un avance importante en la arquitectura de computadoras.

A partir de la década de 1960, el tamaño de los sistemas comenzó a reducirse gracias a la incorporación de circuitos integrados. Un punto clave ocurrió en 1970, cuando Texas Instruments desarrolló el circuito TTL 74181, una ALU de 4 bits capaz de realizar operaciones aritméticas y lógicas como suma, resta, AND, OR y XOR. Este componente simplificó el diseño de las minicomputadoras y se consolidó como un elemento esencial en la computación digital moderna.

Tipos de ALU

Existen diferentes tipos de ALU según el tipo de operaciones que realizan:

- ALU basada en enteros: Ejecuta operaciones aritméticas y lógicas básicas con números enteros, como suma, resta, multiplicación y división.
- ALU de punto flotante (FPU): Diseñada para operar con números decimales, es fundamental en aplicaciones como gráficos y simulaciones científicas.
- ALU de rebanada de bits (bit-slice): Construida a partir de unidades más pequeñas que se combinan para trabajar con mayores tamaños de datos, como 16 o 32 bits.

ALU en CPU y GPU

La implementación de la ALU varía dependiendo del tipo de procesador:

En CPUs (Unidad Central de Procesamiento):

Se caracterizan por tener pocos núcleos, pero altamente complejos. Su procesamiento es secuencial, ejecutando tareas una tras otra. Ofrecen gran versatilidad para tareas generales y decisiones lógicas complejas. Están optimizadas para baja latencia, lo que permite respuestas rápidas en operaciones individuales. Se utilizan principalmente en sistemas operativos, aplicaciones y cálculos diversos.

En GPUs (Unidad de Procesamiento Gráfico):

Cuentan con miles de núcleos más simples. Su procesamiento es paralelo masivo, permitiendo ejecutar múltiples hilos simultáneamente. Están especializadas en operaciones

matemáticas repetitivas sobre grandes volúmenes de datos. Ofrecen alto rendimiento (throughput) y se emplean en renderizado 3D, edición de video, inteligencia artificial y cálculos científicos.

Asignaciones en Verilog

Las instrucciones de asignación bloqueante se asignan mediante `=` y se ejecutan una tras otra en un bloque procedimental. Cada instrucción debe completarse antes de que se ejecute la siguiente. Sin embargo, esto no impedirá la ejecución de instrucciones que se ejecuten en un bloque paralelo.

Utilice asignaciones de bloqueo (`=`) para:

- Lógica combinacional: Bloques `always @(*)` OR `always @(sensitivity_list)` que modelan circuitos combinacionales
- Estímulo del banco de pruebas: Generación secuencial de pruebas en `initial` bloques
- Variables temporales: Cálculos intermedios dentro de un único bloque de procedimiento.

La asignación no bloqueante permite programar asignaciones sin bloquear la ejecución de las instrucciones subsiguientes y se especifica mediante un `<=` símbolo. Es interesante observar que este mismo símbolo se utiliza como operador relacional en expresiones (menor o igual que) y como operador de asignación en el contexto de una asignación no bloqueante.

Utilice asignaciones no bloqueantes (`<=`) para:

- Lógica secuencial (con reloj): En `always @(posedge clk)` bloques que modelan biestables y registros.
- Máquinas de estados: Actualizaciones del registro de estado y salidas registradas
- Etapas de la canalización: Propagación de datos a través de los registros de la canalización

Para la construcción de una ALU debemos basarnos en una estructura organizada y conceptos de diseño lógico presentados en el contexto de computación. El anexo b5 habla sobre esto y como la ALU se establece en el núcleo del procesador encargándose de ejecutar operaciones que definen la capacidad de cálculo de un sistema.

La ALU se clasifica como un componente necesario del Datapath, la organización se divide en:

- input
- output
- memory
- processor
- control

Para hacer una ALU se necesitan los siguientes elementos:

- Diseño lógico: Uso de puertas y ecuaciones lógicas para definir operaciones
- operaciones aritméticas: Implementación de sumadores restadores y comparadores.
- Carry: Manejo de acarreo para acelerar los sumadores básicos
- Multiplexores: Son usados para seleccionar cual de las operaciones realizadas enviará la salida al final

El objetivo de esta actividad fue diseñar e implementar una ALU de 16 bits junto con su respectivo testbench. Para ello, se analizó el Anexo B5 del libro Computer Organization and Design, donde se describe la construcción de una ALU básica para el procesador MIPS. Además, se realizó una investigación complementaria sobre la historia de la ALU, sus tipos y su funcionamiento en CPU y GPU, con el fin de comprender mejor su importancia dentro de la arquitectura de computadoras.

El código realizado en Verilog está basado en la estructura presentada del anexo b5. Este código se diseñó con tres entradas principales: ALUctl (selector de operación), A y B (operandos), y varias salidas representadas mediante buses de datos, incluyendo una señal adicional denominada Zero.

Se utilizó un multiplexor implementado mediante un operador ternario para seleccionar la operación correspondiente según el valor de ALUctl. Posteriormente, se evaluó si el resultado era cero para activar la bandera Zero.

Asimismo, se empleó la estructura case para definir las distintas operaciones. En el caso de la división, se añadió una validación para evitar realizar la operación cuando el divisor es cero, previniendo errores en la ejecución.

La práctica resultó accesible en términos generales. Al inicio, la comprensión del material teórico representó un reto, pero con varias revisiones se logró entender la lógica de diseño de la ALU. La implementación en Verilog permitió reforzar estos conceptos y trasladarlos a un contexto práctico. En conjunto, la actividad fue útil para comprender el funcionamiento interno de los procesadores y el papel fundamental de la ALU.

Referencias bibliográficas:

Arithmetic Logic Units (ALU): An introduction. (s/f). Arith-matic.com. Recuperado el 16 de marzo de 2026, de <https://arith-matic.com/notebook/arithmetic-logic-unit-introduction>

(Corsair Gaming, 2024)

Corsair Gaming. (2024, agosto 14). CPU vs GPU vs NPU: ¿Cuál es la diferencia?

Corsair.com.

<https://www.corsair.com/es/es/explorer/diy-builder/power-supply-units/cpu-vs-gpu-vs-npu-whats-the-difference/>

Verilog Blocking & non-Blocking. (2018, febrero 9). ChipVerify.

<https://www.chipverify.com/verilog/verilog-blocking-non-blocking-statements>